CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 2 Report

Team Members: David Rice

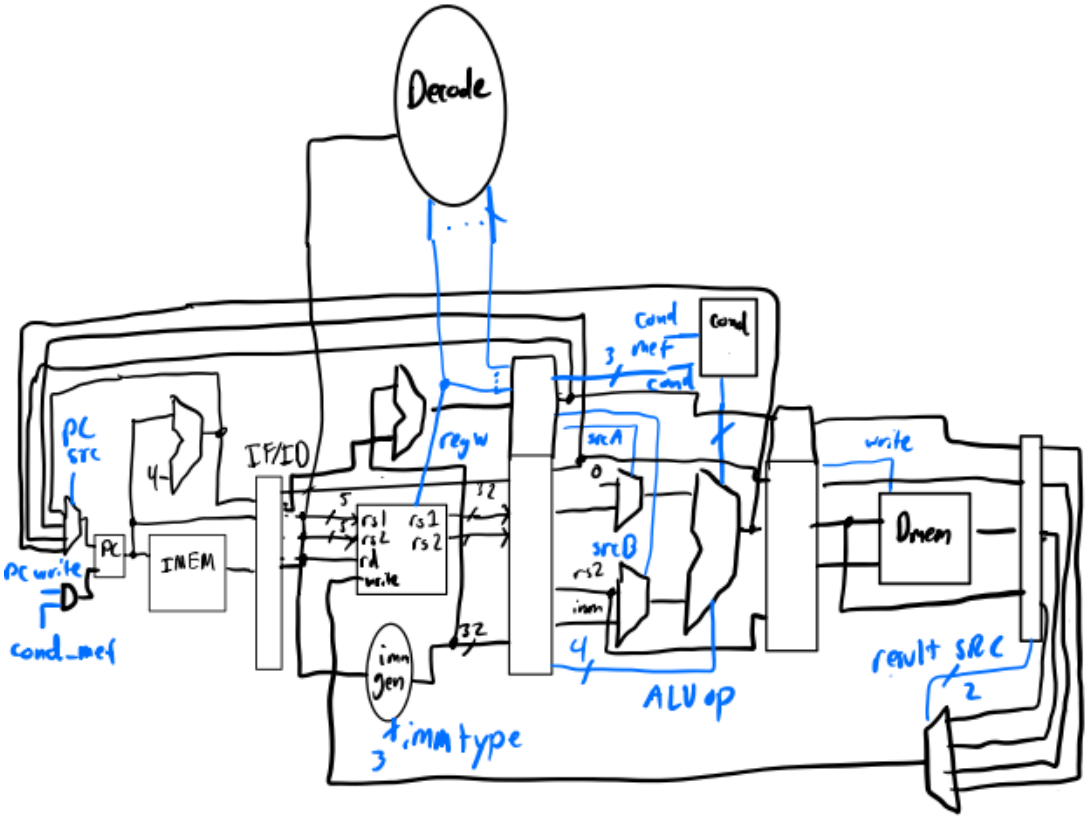
Nick Jund

Project Teams Group #: 6

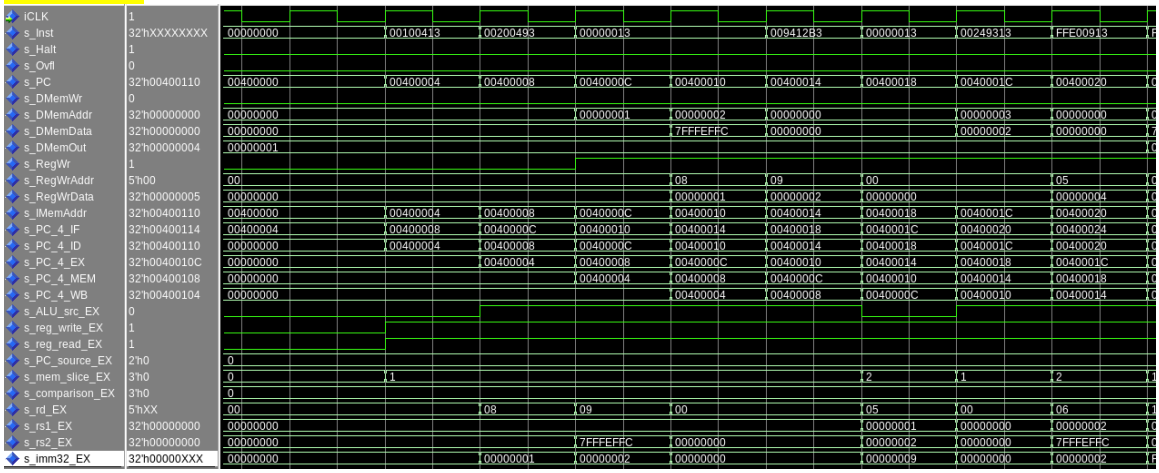
Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[1.a] Come up with a global list of the datapath values and control signals that are required during each pipeline stage.
Attached in spreadsheet

[1.b.ii] high-level schematic drawing of the interconnection between components.

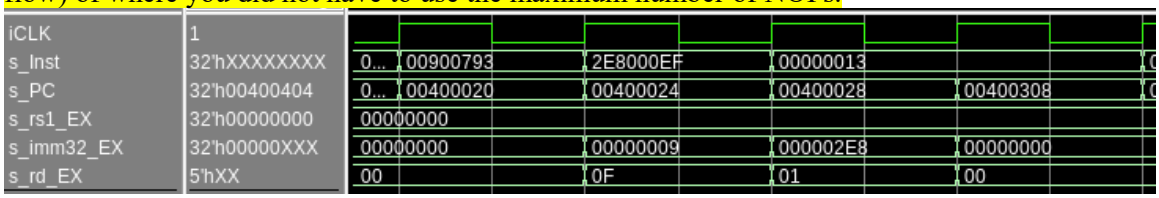


[1.c.i] include an annotated waveform in your writeup and provide a short discussion of result correctness.

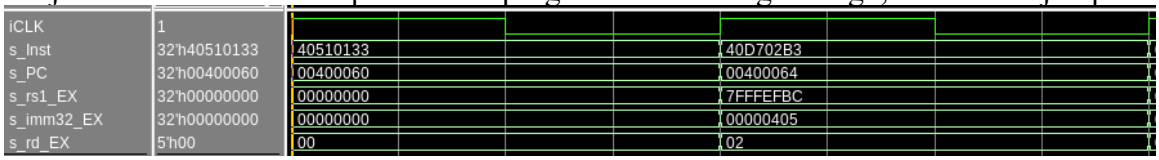


S_PC_4 is shown for all stages and it can be seen propegating. s_Inst is the active instruction. Whenever the instruction is 0x00000013 that represents a nop. The fist 4 instructions of the testbench are addi s0, zero, 1 addi s1, zero 2 and two nops the register sources and immediates in the execution stage can be seen and the corespond with the instruction but delayed because they happen on different stages.

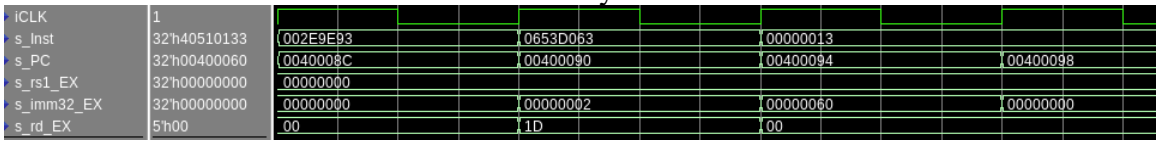
[1.c.ii] Include an annotated waveform in your writeup of two iterations or recursions of these programs executing correctly and provide a short discussion of result correctness. In your waveform and annotation, provide 3 different examples (at least one data-flow and one control-flow) of where you did not have to use the maximum number of NOPs.



This screenshot shows the instructions li a5 9 and jal ra, merge_sort without any NOPs in between. The li, which becomes an addi can be seen adding 0, s_rs1_EX, and 9, s_imm32_EX, and putting the result in a5, or x15 as seen in s_rd_EX. No NOPs are needed between these instructions because they do not use the same registers. However following the jal there are 2 NOPs to prevent the program from falling through, rather than jump.



This screenshot shows sub sp sp t0 and sub t0 a4, a3 being run in succession without NOPs as the the first sub uses t0 before it is written by the next sub.

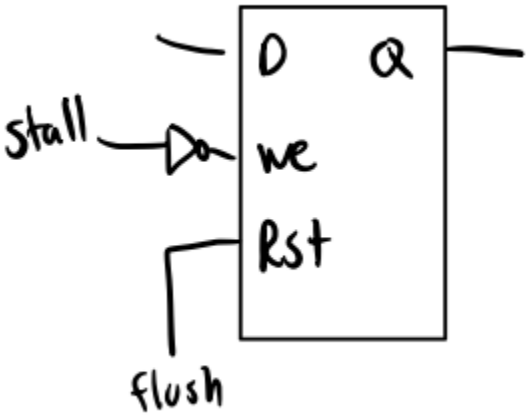


This screenshot shows slli t4, t4, 2 and bge t2 t0 end_copy_L. Once again no NOPs are needed between these registes as they use non-overlapping registers. The bge is followed by two NOPs as to not load the next instruction in case the branch is taken.

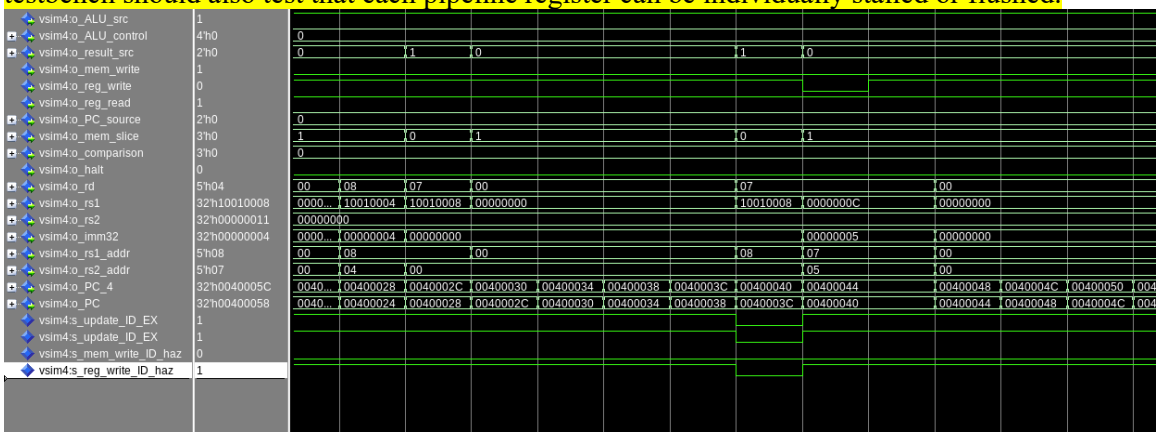
[1.d] report the maximum frequency your software-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

58.19 MHz with a critical path from the read port of our (negative edge) register file to the ID/EX register.

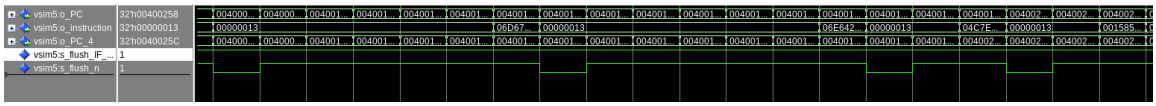
[2.a.ii] Draw a simple schematic showing how you could implement stalling and flushing operations given an ideal N-bit register.



[2.a.iii] Create a testbench that instantiates all four of the registers in a single design. Show that values that are stored in the initial IF/ID register are available as expected four cycles later, and that new values can be inserted into the pipeline every single cycle. Most importantly, this testbench should also test that each pipeline register can be individually stalled or flushed.



From data hazard test the ID/EX's write values stall when update goes low (opposite of stall).



When the flush goes low the output gets reset to a NOP in IF/ID for flushing.

[2.b.i] list which instructions produce values, and what signals (i.e., bus names) in the pipeline these correspond to.
Included in the spreadsheet

[2.b.ii] List which of these same instructions consume values, and what signals in the pipeline these correspond to.
Included in the spreadsheet

[2.b.iii] generalized list of potential data dependencies. From this generalized list, select those dependencies that can be forwarded (write down the corresponding pipeline stages that will be forwarding and receiving the data), and those dependencies that will require hazard stalls.
Included in spreadsheet

[2.b.iv] global list of the datapath values and control signals that are required during each pipeline stage

Included in spreadsheet

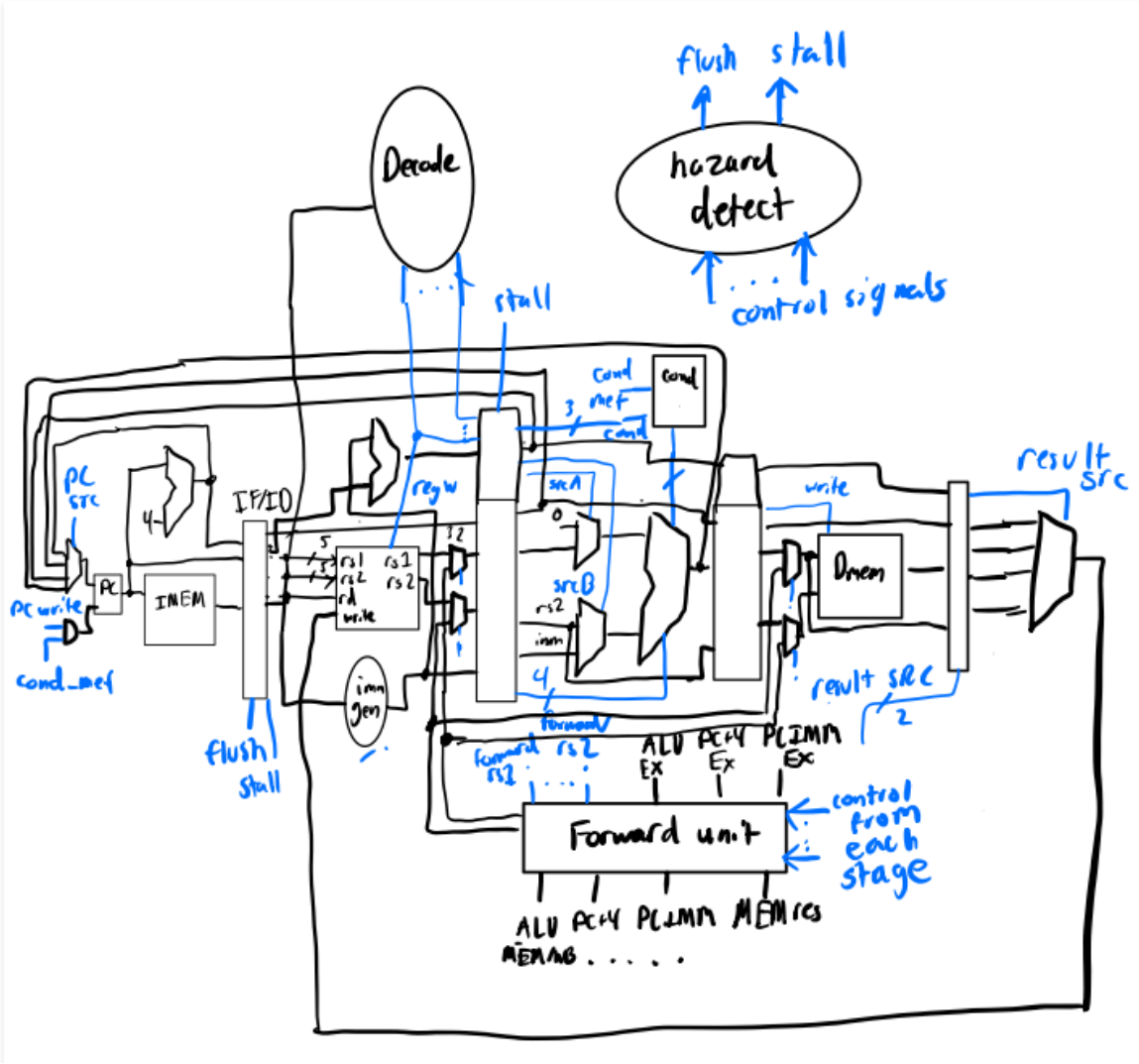
[2.c.i] list all instructions that may result in a non-sequential PC update and in which pipeline stage that update occurs.

All the branch instructions, jal, and jalr can cause a non-sequential PC update at the EX stage.

[2.c.ii] For these instructions, list which stages need to be stalled and which stages need to be squashed/flushed relative to the stage each of these instructions is in.

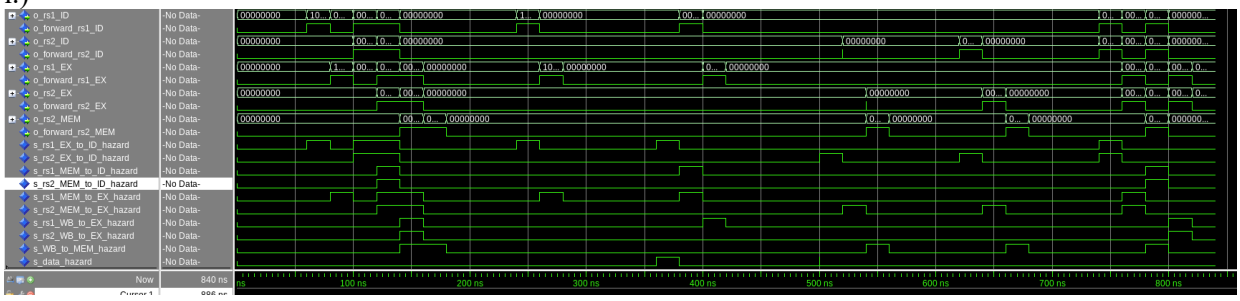
The only thing it has to flush is the previous stage's control register. Stalling is not necessary.

[2.d] implement the hardware-scheduled pipeline using only structural VHDL. As with the previous processors that you have implemented, start with a high-level schematic drawing of the interconnection between components.



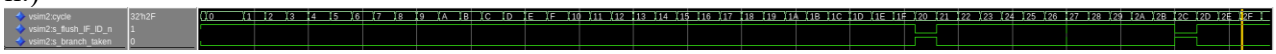
[2.e – i, ii, and iii] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

i.)



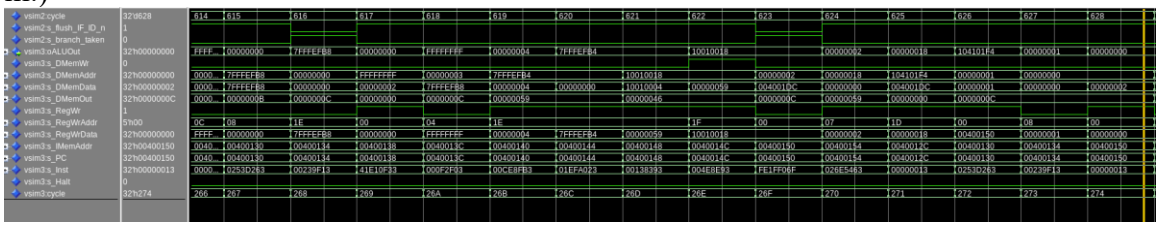
For the data hazard case, the only time a true data hazard occurs (s_data_hazard) is with a load followed by an arithmetic instruction, as that is the only one that cannot be resolved via forwarding. The other cases detect a hazard and set the proper forwarding case.

ii.)



For the control hazard case we used our control flow test from project 1 as it hits every control hazard/not taken branch. When the branches are taken, the flush signal becomes active to flush the previous stage (active low).

iii.)



This is an excerpt from mergesort. All the signals for the testbench are well defined and it has no errors simulating.

```
Testing file: proj/riscv/addiseq.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 11
Processor Cycles: 15
CPI:  1.36
Results in: output/addiseq.s
```

```
Testing file: proj/riscv/lab3Seq.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 22
Processor Cycles: 26
CPI:  1.18
Results in: output/lab3Seq.s
```

```
bash-5.1$ ./3810 tf.sh test proj/riscv/*.s
Testing file: proj/riscv/simplebranch.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 12
Processor Cycles: 26
CPI:  2.17
Results in: output/simplebranch.s
```

```
Testing file: proj/riscv/fibonacci.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 331
Processor Cycles: 426
CPI:  1.29
Results in: output/fibonacci.s
Testing file: proj/riscv/Proj1_base_test.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 39
Processor Cycles: 45
CPI:  1.15
Results in: output/Proj1_base_test.s
Testing file: proj/riscv/Proj1_cf_test.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 731
Processor Cycles: 883
CPI:  1.21
Results in: output/Proj1_cf_test.s
```

```
Failed to remove output/data_forward_test.s. Is questasim open?
Saving instead to "output/data_forward_test.s_1"
Testing file: proj/riscv/data_forward_test.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 36
Processor Cycles: 43
CPI:  1.19
Results in: output/data_forward_test.s_1
------------------------------------------------
```

```
Testing file: proj/riscv/Proj1_mergesort.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 1634
Processor Cycles: 2189
CPI:  1.34
Results in: output/Proj1_mergesort.s
Testing file: proj/riscv/grendel.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 2129
Processor Cycles: 3008
CPI:  1.41
Results in: output/grendel.s_1
```

[2.e.i] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

Most of the possible hazards (plus double hazards) were tested. However a few were left out as they are so similar to a few other ones (like arithmetic to auipc, and arithmetic to jal/jalr).

[2.e.ii] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

For the control flow hazards we used our control flow test from project 1 as that uses every instruction that could cause a hazard, and tests each combination of taken/not taken (if relevant).

[2.f] report the maximum frequency your hardware-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

51.60 MHz which goes through the ALU to the comparison unit in fetch logic, and then the flush signal for control hazard detection.